

Expona Recommended Module

InsaneSDD Change Impact and Delivery Risk Explorer

Original Expona name:

Cross-Repo Dependency Visualization and Impact Analysis Module

These names refer to the **same single module**, not two micro-frontends.

Purpose

Visualize how a changed requirement or code change affects multiple repositories and delivery artifacts, allowing teams to identify dependencies, conflicts and delivery risks before implementation or merging.

Target users and buyers

- Engineering managers
- Software architects
- Frontend leads
- Technical leads managing multi-repository projects

Problem solved

Multi-repository development can hide dependencies and integration conflicts. Automated branches and pull requests may increase this risk when teams cannot easily understand the complete impact of a change.

The module provides a visual, explainable view of:

- What is affected
- Why it is affected
- How risk propagates
- What needs attention before the change is merge-ready

Relationship to InsaneSDD

The module is intended as a **complementary extension** to InsaneSDD capabilities such as:

- Specification analysis
- Multi-repository branch intelligence
- Automated pull-request creation
- Conflict detection
- Requirements-to-delivery traceability

It should not be presented as proof that InsaneSDD lacks impact analysis. The public information does not confirm the internal feature depth. It should be described as a concept that improves the visibility and governance of the publicly demonstrated workflow.

Pasted markdown

Complete user workflow

1. Select a changed requirement.
2. Load the related InsaneSDD project context.
3. Display the affected:
 4. Epics
 5. Stories
 6. Tasks
 7. Repositories
 8. Branches
 9. Pull requests
10. Tests
11. Map dependencies between the affected artifacts.
12. Visualize how impact and risk propagate through those dependencies.
13. Highlight:
 14. Cross-repository conflicts
 15. Conflicting active branches or pull requests
 16. Missing test coverage
 17. Unresolved approvals
18. High-risk affected artifacts
19. Allow users to drill into an artifact and inspect:
 20. Why it was identified as affected
 21. Which relationship caused the impact
 22. Why the AI assigned its risk level
 23. What evidence supports the finding
24. Show traceability from the changed specification to merge readiness.
25. Suggest mitigation actions for identified risks.

26. Produce an impact and delivery-risk view for reviewers before implementation or merge.

Core product capabilities

Change input

- Select a changed requirement
- View the original requirement context
- Start impact analysis from that change

Cross-repository dependency mapping

- Show relationships across multiple repositories
- Connect requirements with epics, stories and tasks
- Connect tasks with repositories, branches and pull requests
- Connect affected work with tests
- Show direct and indirect dependencies

Interactive impact visualization

- Interactive dependency graph
- Expand and collapse dependency paths
- Filter by artifact type
- Drill into individual artifacts
- Highlight affected dependency paths
- Change-impact heatmaps
- Risk propagation visualization

Delivery-risk analysis

- Identify high-risk changes
- Detect integration conflicts
- Surface conflicting branches or pull requests
- Highlight missing test coverage
- Highlight unresolved approvals
- Show risk severity
- Explain why each risk was assigned
- Suggest mitigation actions

Traceability

Show an evidence trail across:

Changed specification → epic → story → task → repository → branch → pull request → test → merge readiness

Review and governance

- Role-based access to sensitive repository information
- Review risk findings
- Inspect AI reasoning and evidence
- Track unresolved approvals

- Maintain an audit trail of dependency and impact changes

Frontend capabilities demonstrated

- Complex application state management
- Interactive graph visualization
- Large-data rendering
- Graph and table synchronization
- Drill-down navigation
- Search, filtering and grouping
- Change-impact heatmaps
- Real-time or simulated event updates
- WebSocket or event-stream handling
- Responsive enterprise UI
- Accessibility
- Keyboard-accessible graph alternatives
- Performance optimization
- Role-based UI permissions
- Design-system architecture
- Loading, empty, error and partial-data states
- Frontend testing strategy

Backend and AI integration requirements

- InsaneSDD specification-analysis integration
- InsaneSDD branch-intelligence integration
- Repository metadata provider
- Branch and pull-request provider
- Test-coverage provider
- Dependency-graph generation
- AI-assisted predictive impact analysis
- AI-generated risk classification
- Explainable evidence for impact findings
- Conflict-detection input
- Audit-event storage

For the interview POC, these integrations can be represented through deterministic mocked providers and simulated InsaneSDD project data.

Security and audit capabilities

- Access control for sensitive repository data
- Role-based permissions
- Audit trail of dependency changes
- Audit trail of risk findings
- Evidence behind AI-generated findings
- Visibility of pending approvals
- Traceability from requirement to merge readiness

Presentation statement

"A concept demonstrating how a frontend experience could make multi-repository, specification-driven delivery easier to understand and govern."

Do not present it as:

"I discovered that InfoBeans does not have this feature."

Exact scope summary

The Expona recommendation is one module containing:

Changed requirement → cross-repository dependency graph → affected delivery artifacts → risk propagation → conflicts → missing tests → unresolved approvals → explainable evidence → specification-to-merge-readiness traceability

This is the agreed first POC direction.